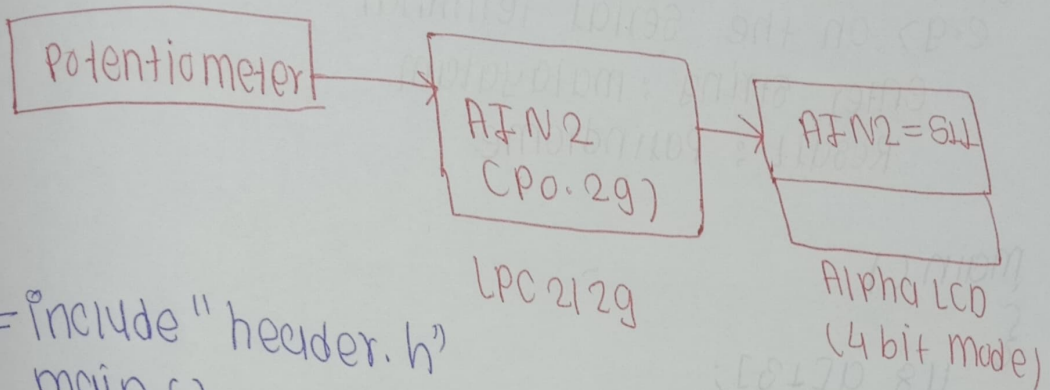


Assignment

- Write an ECP to scan sensor value from AIN2 (CH2) of ADC & Print it on the alpha lcd.



```
#include "header.h"
main()
```

```
{
    unsigned int temp;
    float vout;
    adc_init();
    uart0_init(9600);
    uart0_tx_string("ADC Test\r\n");
    while(1)
    {
        temp = adc_read(2);
        vout = (temp * 3.3) / 1023;
        uart0_integer(temp);
        uart0_tx(' ');
        uart0_float(vout);
        uart0_tx_string("\r\n");
        delay_ms(100);
    }
}
```

Convert ADC to Vtg

② Write an ECP, to receive a string serially using UART0, check it is Palindrome or not & print result on the led.

e.g> on the serial terminal

enter string : malayalam

Result : Palindrome

main()

{

US a[13];

U32 i, j, l;

UART0_Init(9600);

while(1)

{

UART0_tx_string("\r\n enter the string");

UART0_tx_string(a, 12);

UART0_tx_string("\r\n");

UART0_tx_string(a);

for(i=0; a[i]; i++);

for(i=0, j=i-1; i<j; i++, j--)

{

if(a[i] != a[j])

{

UART0_tx_string("\r\n not Palindrome");

break;

}

if(a[i] == a[j])

UART0_tx_string("\r\n Palindrome");

}

}

Q8) check given string is palindrome or not using
uart0 - interrupt.

```
/* uart0 - interrupt */  
#include <ip21xx.h>  
#include "header.h"  
u32 flag, i, num;  
extern s8 arr[13];  
void UART0_Handlee(void) - irq  
{  
    num = U0IR & 0x0E;  
    if (num == 0x04)  
    {  
        arr[i++] = U0RBR;  
    }  
    if case[i-1] == '\0'  
    {  
        case[i-1] = '\0';  
        flag = 1;  
        i = 0;  
    }  
    if (num == 0x02)  
    {  
        VICVectAddr = 0;  
    }  
}  
void config_vic_for_uart0(void)  
{  
    VICIntSelect = 0;  
    VICVectAddr0 = (u32) UART0_Handlee;  
}
```



```
VICInt Enable = C1 << 16;
```

```
}
```

```
void en_uart0_interrupt(void)
```

```
{
```

```
    UOIER = 1;
```

```
}
```

```
/* uart0 - interrupt - main */ /* Palindrome */
```

```
#include "header.h"
```

```
extern unsigned flag;
```

```
char arr[13];
```

```
main()
```

```
{
```

```
    unsigned i, j;
```

```
    config_vic_for_uart0();
```

```
    en_uart0_interrupt();
```

```
    uart0_init(9600);
```

```
    uart0_tx_string("enter the char:\n");
```

```
    while(1)
```

```
{
```

```
    if (flag == 1)
```

```
{
```

```
        flag = 0;
```

```
        uart0_tx_string(arr);
```

```
        for (i = 0; arr[i]; i++);
```

```
        for (i = 0, j = i - 1; i < j; i++, j--)
```

```
{
```

```
            if (arr[i] != arr[j])
```

```
                break;
```

```
}
```

```

if (arr[i] == arr[j])
    uart0 - tx - string ("In given string is pal...");
    uart0 - tx - string ("In given string is not
        Pal...");
    uart0 - tx - string ("\n");
}
}
}

```

1 ★ reverse ★ /

```

#include "header.h"
extern u32 flag;
s8 arr[13];
main()
{
    u32 i, j, temp;
    config_vic_for_uart0();
    en_uart0_interrupt();
    uart0 - init(9600);
    uart0 - tx - string ("enter the char:\n");
    while (1)
    {
        if (flag == 1)
        {
            flag = 0;
            uart0 - tx - string (arr);
            uart0 - tx - string ("\n");
            for (i = 0; arr[i]; i++);
            for (i = 0, j = i - 1; i < j; j--, i++)
            {
                temp = arr[i];
            }
        }
    }
}

```



```
arr[i] = arr[j];  
arr[j] = temp;
```

```
}  
uart0 - tx - string (arr);  
uart0 - tx - string ("\n");
```

```
}  
}  
}
```

4) implement password authentication pgm using
uart0

```
#include "header.h"  
main ()
```

```
{  
    char a[12], b[12] = "Aarti";  
    int i;  
    uart0 - init(9600);  
    while(1)  
    {  
        uart0 - tx - string ("enter the password\n");  
        uart0 - rx - string (a, 11);  
        for (i=0; b[i]; i++)  
            uart0 - tx ('*');  
        for (i=0; b[i]; i++)  
        {  
            if (b[i] != a[i])  
                break;  
        }  
        if (a[i] == b[i])  
        {  
            uart0 - tx - string ("\n correct  
password");  
        }  
    }  
}
```




```
while (1)
```

```
{
```

```
    if (flag == 1)
```

```
    {
```

```
        flag = 0;
```

```
        switch (temp)
```

```
        {
```

```
            case 'a':
```

```
            {
```

```
                IOCLR0 = LED;
```

```
                delay_ms(200);
```

```
                IOSET0 = LED;
```

```
                delay_ms(200);
```

```
                break;
```

```
            }
```

```
            case 'b':
```

```
            {
```

```
                for (i = 16; i <= 23; i++)
```

```
                {
```

```
                    IOCLR0 = 1 << i;
```

```
                    delay_ms(200);
```

```
                    IOSET0 = 1 << i;
```

```
                    delay_ms(200);
```

```
                }
```

```
                break;
```

```
            }
```

```
            default:
```

```
            {
```

```
                uart0_tx_string("incorrect option  
input\n");
```

```
                break;
```

```
            }  
        }  
    }  
}
```

Q6] write an ECP, to toggle an LED everytime when the EINT3 is triggered.

LED1 $\rightarrow$ P0.17

EINT3 $\rightarrow$ P0.20

NOTE : main fun has to blink LED2 at the rate of 1 sec.

```
/* einto.c */
```

```
#include <ipc21xx.h>
```

```
#include "header.h"
```

```
#define LED (1<<17)
```

```
void EINT0_Handlere (void) __irq
```

```
{
```

```
    static int flag;
```

```
    /* LED toggle logic */
```

```
    flag ^= 1 // toggle 0th bit
```

```
    if (flag)
```

```
        IOCLR0 = LED; // turn on AL LED
```

```
    else
```

```
        IOSET0 = LED; // turn off AL LED
```

```
    EXTINT0 = 1; // clear EINT0 inter flag
```

```
    VICVectAddr = 0; // to finish isr() execution
```

```
}
```

```
void config_vic_for_einto (void)
```

```
{
```

```
    VICIntSelect = 0;
```

```
    VICVectCntl0 = 14 | (1<<5); // Assign inter no to slot 0
```

```
    VICVectAddr0 = (432) EINT0_Handlere; // Assign isr Addr to slot 0
```

```

VICIntEnable |= (1 << 14);
}
void config_eint0(void)
{
    EXTPOLAR = 0; // eint0 is active low
    EXTMODE = 1; // eint0 is edge triggered
    PINSEL1 |= 1; // P0.16 --> eint0
    IODIR0 |= LED; // P0.17 --> is o/p
}

```

```

/* main_eint0.c */
#include <lpc21xx.h>
#include "header.h"
#define LED (1 << 17)
u32 cout;
main()
{
    IODIR0 |= LED;
    Config_Vic_for_eint0();
    Config_eint0();
    while(1)
    {
        IOCLR0 = LED;
        delay_ms(1000);
        IOSET0 = LED;
        delay_ms(1000);
    }
}

```



```
VICVectAddr=0;
```

```
}
```

```
void Config_vic_for_uart0 (void)
```

```
{
```

```
    VICIntSelect=0;
```

```
    VICVectAddr= (432) UART0_Handlerr;
```

```
    VICVectCntl0=6+(1<<5);
```

```
    VICIntEnable= (1<<6);
```

```
}
```

```
void en_uart0_interrupt (void)
```

```
{
```

```
    UOIER = 1;
```

```
}
```

```
/* main */
```

```
#include "header.h"
```

```
u32 flag, j;
```

```
s8 arr[17];
```

```
main()
```

```
{
```

```
    Config_vic_for_uart0();
```

```
    en_uart0_interrupt();
```

```
    uart0_init(9600);
```

```
    lcd_init();
```

```
    uart0_tx_string("enter the string:\n");
```

```
    while(1)
```

```
    {
```

```
        if(flag==1)
```

```
        {
```

```
            flag=0;
```

```
            uart0_tx_string(arr);
```

```
            lcd_cmd(0x01);
```

```
            for(j=0; arr[j]; j++)
```

```

{
    lcd - data (arr[i]);
}
uart0 - tx - string ("r\n");
}
}
}

```

q) Implement the given fun for uart0.

```

1) void uart0 - tx - integer (int);
2) void uart0 - tx - float (float);
e.g > uart0 - tx - integer (1234);
      uart0 - tx - float (12.34567);

```

integer :-

```
#include <1PC21xx.h>
```

```
#include "header.h"
```

```
void uart0 - tx - integer (s32 num)
```

```
{
```

```
    char a[10];
```

```
    int i=0;
```

```
    if (num==0)
```

```
    {
```

```
        uart0 - tx ('0');
```

```
        return;
```

```
    }
```

```
    if (num < 0)
```

```
    {
```

```
        uart0 - tx ('-');
```

```
        num = -num;
```

```
    }
```

```
    while (num > 0)
```

```
    {
```



```

aci] = num%10 + 48;
num = num/10;
i++;
}
for (i=i-1; i>=0; i--)
    werto -tx (aci];
}

```

```

/ ★ main ★ /
#include "header.h"
main()
{
    werto_init(9600);
    werto -tx -integer(125);
}

```

```

float
#include <stdio.h>
#include "header.h"
void werto -tx -float (float f)
{
    int i;
    if (f < 0)
    {
        werto -tx ('-');
        f = -f;
    }
    if (f == 0)
    {
        werto -tx -string("0.0");
        return;
    }
}

```

```
i=f;
```

```
uart0_tx_integer(i);
```

```
uart0_tx('.');
```

```
i=(f-i)*1000000;
```

```
uart0_tx_integer(i);
```

```
}
```

```
/* main */
```

```
#include "header.h"
```

```
main()
```

```
{
```

```
uart0_init(9600);
```

```
uart0_tx_float(12.34);
```

```
}
```

```
/* RFID main.c */
```

```
#include <1pc21xx.h>
```

```
#include "header.h"
```

```
int main()
```

```
{
```

```
char a[13];
```

```
uart0_init(9600);
```

```
lcd_init();
```

```
while(1)
```

```
{
```

```
uart0_tx_string(a,12);
```

```
uart0_tx_string(a);
```

```
uart0_tx_string("rin");
```

```
lcd_cmd(0x1);
```

```
lcd_string(a);
```

```
}
```

```

1 *uart0 - Drive.c* /
#include <ipc21xx.h>
#include "header.h"
#define RDR ((VOLSR & 1))
#define u8 unsigned char
#define u32 unsigned int
#define THRE ((VOLSR >> 5) & 1)

1 *uart0 init* /

```

```

void uart0_init(unsigned int baud)
{
    unsigned int result = 0;
    int arr = { 15, 60, 80, 15, 15 };
    unsigned int pclk = arr[VPRDIV] * 1000000;
    result = pclk / (16 * baud);
    PINSEL0 |= 5;
    VOLCR = 0x83;
    UODLL = result & 0xFF;
    UODLM = (result >> 8) & 0xFF;
    VOLCR = 0x03;
}

```

```

1 *uart transmit* /
void uart0_tx(unsigned char data)
{
    UOTHR = data;
    while (THRE == 0)
    {
    }
}

```



```
/* tx string fun */  
void uarto_tx_string(char *ptr)  
{
```

```
    while(*ptr)
```

```
{
```

```
    UOTHR = *ptr;
```

```
    while (CTHR == 0);
```

```
    ptr++;
```

```
}
```

```
}
```

```
/* uarto receives */
```

```
unsigned char uarto_rx(void)
```

```
{
```

```
    while ((UOLSR & 1) == 0);
```

```
    return UORBR;
```

```
}
```

```
/* uarto tx-integer fun */
```

```
void uarto_tx_integer(int num)
```

```
{
```

```
    int a[10], i;
```

```
    if (num == 0)
```

```
{
```

```
        uarto_tx('0');
```

```
        return;
```

```
}
```

```
    if (num < 0)
```

```
{
```

```
        uarto_tx('-');
```

```
        num = -num;
```

```

i=0;
while (num > 0)
{
    a[i] = (num % 10 + 48);
    num /= 10;
    i++;
}
for (i = i-1; i >= 0; i--)
    printf("%c", a[i]);
}

```

~~/* uart0 receiver */~~

~~unsigned char uart0_rx(void)~~

~~S~~

~~/* ex_string */~~

~~void uart0_rx_string(char *pt, int max_byte)~~

~~{~~

~~int i;~~

~~for (i=0; i < max_byte; i++)~~

~~{~~

~~while (RDR == 0);~~

~~pt[i] = UORBR;~~

~~if (pt[i] == '\n')~~

~~break;~~

~~}~~

~~pt[i] = '\0';~~

~~}~~

1 * float function * /
void uarto_tx_float (float F)

```
{  
    int n=F;  
    uarto_tx_integer(n);  
    uarto_tx('.');  
    n = (F-n)*1000000;  
    if(n<0)  
    {  
        uarto_tx('-');  
        n=-n;  
        uarto_tx_integer(n);  
    }  
}
```

Q.10] write an ECP to receive a character repeatedly from serial terminal & retransmit its ASCII value in 3 digits.

eg: on the serial terminal

Enter char: A

ASCII: 65

main c)

```
{  
    unsigned char temp; // us temp, temp;  
    uarto_init(9600);  
    while(1)  
    {  
        uarto_tx_string("In enter char: ");  
        temp = uarto_rx();  
    }  
}
```



```

uart0 - tx (temp); // loopback
uart0 - tx cstring ("In ASCII:");
1 + temp;
uart0 - tx ((temp/10)%10+48);
uart0 - tx ((temp%10)+48);
}

```

O/P :- Enter char : A
 ASCII : 65

Q27 WAP ECP to receive a character repeatedly from serial terminal & retransmit its ascii value in 3 digits decimal & hexadecimal
 e.g. on the serial terminal

Enter char : A
 ASCII : 065 0x41

Enter char

Implement $\rightarrow$ void uart0 - tx hex (int);
 $\downarrow$
num(65/'A')

num(65/'A')

$\downarrow$

Convert dec to Hex

Send '0'

'x'

'4'

'1'

65

0100 0001

$\downarrow$
4

$\downarrow$
1

#include <LPC21xx.h>

#include "header.h"

main()

{

us temp, temp1;

uart0 - init (9600);

while (1)

```
{  
    clrto - tx - string ("r\n enter the char:");  
    temp = clrto - &xc1;  
    clrto - tx ctemp;  
    clrto - tx - string ("r\n char ASCII:");  
    temp1 = temp;  
    clrto - tx c(temp / 100) + 48);  
    us num = temp / 10;  
    clrto - tx c(num % 10) + 48);  
    clrto - tx c(temp / 10 % 10) + 48);  
    clrto - tx - string ("r\n char in hexa:");  
    clrto - tx c(temp / 16) + 48);  
    clrto - tx c(temp % 16) + 48);  
}
```

Q117) WAFEC to receive a char from the serial terminal & loopback its binary.

eg:

Enter char: A

Binary: 01000001

Enter char:

```
#include <stdio.h>
```

```
#include "header.h"
```

```
main ()
```

```
{
```

```
    us temp, num;
```

```
    int i;
```

uart0-init
led-init
while(
Σ uart0
ten
uc

}
}